

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Y. Jithendra Reddy	Reg. No.:	192525303
Section / Batch:	AIML (2025)	Date:	9-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I, Y. Jithendra Reddy (192525303) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jithendra Reddy

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1

Search Optimization in Digital Archives (Binary Search) A digital archive stores millions of documents in sorted order and requires fast retrieval

100 marks

Analyze how Binary Search improves efficiency. Identify requirements such as indexing and sorted data. Apply divide and conquer concepts to explain the process. Analyze time complexity and justify its suitability.

Search Optimization in Digital Archives Using Binary Search

A digital archive stores millions of documents in sorted order. Binary Search is highly suitable because it finds a required document by repeatedly reducing the search space into half.

1. How Binary Search improves efficiency

In a Linear Search, documents are checked one by one. For millions of documents, this can require millions of comparisons.

Binary Search checks the middle document and eliminates half of the remaining documents after every comparison.

For example, for 1,000,000 documents:

Linear Search → up to 1,000,000 comparisons

Binary Search → about 20 comparisons, because $\log_2 1,000,000 \approx 20$

Therefore, Binary Search provides much faster retrieval.

2. Requirements

Sorted data:

The documents must be sorted according to a searchable key such as:

Document ID

Title

Date

Author name

Indexing:

An index can store searchable keys and references to document locations. Binary Search can operate on this sorted index rather than searching the complete documents.

Random access:

The system should be able to directly access the middle element efficiently.

3. Divide and Conquer Process

Binary Search follows the Divide and Conquer technique:

Divide: Find the middle element of the search range.

Compare: Compare the required key with the middle element.

Conquer:

If equal → document is found.

If key is smaller → search the left half.

If key is larger → search the right half.

Repeat until the document is found or the range becomes empty.

Example:

Documents: [10, 20, 30, 40, 50, 60, 70]

Search key: 60

Middle = 40 → 60 > 40 → search right half

Middle = 60 → Found

Only 2 comparisons are required.

4. Algorithm

BinarySearch(A, key):

low = 0

high = n - 1

while low <= high:

mid = (low + high) // 2

if A[mid] == key:

return mid

else if key < A[mid]:

high = mid - 1

else:

low = mid + 1

return -1

5. Time and Space Complexity

Case

Time Complexity

Best Case

$O(1)$

Average Case

$O(\log n)$

Worst Case

$O(\log n)$

Space – Iterative

$O(1)$

Space – Recursive

$O(\log n)$

The recurrence relation is:

Therefore,

6. Suitability for Digital Archives

Binary Search is suitable because digital archives contain very large amounts of data. Its logarithmic complexity means the number of comparisons increases very slowly even when **millions of documents are added**.

Conclusion: Binary Search combined with sorted data and proper indexing provides fast, scalable, and efficient document retrieval. It is therefore an excellent search technique for large digital archives.

Your Code

```
1 # Binary Search for Digital Archive
2
3 def binary_search(documents, target):
4     low = 0
5     high = len(documents) - 1
6
7     while low <= high:
8         mid = (low + high) // 2
9
10        if documents[mid] == target:
11            return mid
12        elif documents[mid] < target:
13            low = mid + 1
14        else:
15            high = mid - 1
16
17    return -1
18
19
20 # Input
21 documents = list(map(int, input("Enter sorted document IDs: ").split()))
22 target = int(input("Enter document ID to search: "))
23
24 # Search
25 result = binary_search(documents, target)
26
27 # Output
28 if result != -1:
29     print("Document found at index:", result)
30 else:
31     print("Document not found")
```

Input

```
10 20 30 40 50 60 70
60
```

Output

```
Enter sorted document IDs: Enter
document ID to search: Document
found at index: 5
```

